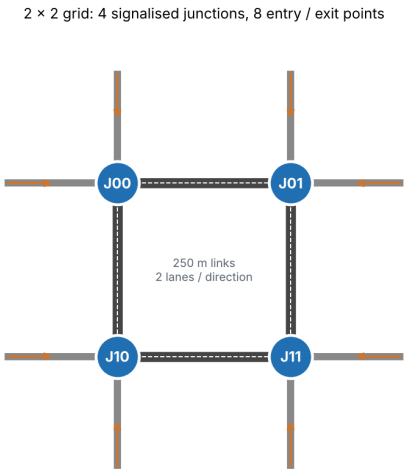
Traffic-Signal Coordination with Multi-Agent Deep Q-Networks

Project 10 - Real-World RL Homework Projects, Unit 5

Multiple agents control the signals of four interconnected intersections to reduce average waiting time.



Control approaches	Fixed-time (baseline) vs independent Multi-Agent DQN
Simulation backend for results	built-in Python mesoscopic simulator (NOT SUMO)
DQN implementation used	NumPy (identical algorithm; PyTorch version also provided)
Date generated	07 November 2026
Contents	Problem formulation | Architecture | Complete Python code | Training methodology | Learning curves &

IMPORTANT: the numbers in this document were produced with the project's built-in Python mesoscopic simulator (tsc/minisim_env.py) because SUMO, TraCI and PyTorch were not available in the environment that built this report. The SUMO code path (tsc/sumo_env.py, tsc/network.py, tsc/agents/dqn_torch.py) is complete but has NOT been executed to produce these figures. Running 'python run_all.py --backend sumo' on a machine with SUMO 1.28 regenerates every table, figure, slide and this PDF from real SUMO runs.

Table of Contents

Deliverable 1 - One-Page Problem Formulation 4

Project Architecture 5

Data flow . . . 5
Project layout . . . 5
Simplified or simulated components (read this) . . . 5

Step-by-Step Execution Guide (VS Code, Windows) 6

Deliverable 2 - Complete Python Implementation 7

run_all.py . . . 7
check_setup.py . . . 8
tsc/config.py . . . 9
tsc/demand.py . . . 10
tsc/minisim_env.py . . . 11
tsc/network.py . . . 13
tsc/plots.py . . . 14
tsc/runner.py . . . 15
tsc/sumo_env.py . . . 17
tsc/agents/__init__.py . . . 19
tsc/agents/dqn_numpy.py . . . 19
tsc/agents/dqn_torch.py . . . 20
tsc/agents/replay.py . . . 21
tsc/__init__.py . . . 21
requirements.txt . . . 21

Training Methodology 23

Reward and state design rationale . . . 23

Deliverable 3 - Learning Curves and Result Tables 24

Table 1 - Test results, demand scale 1.0, mean +/- std over 10 unseen seeds . . . 24
Table 2 - All evaluated controllers and demand levels (test seeds) . . . 25
Table 3 - Per-seed test results (demand scale 1) . . . 25
Interpretation . . . 25

Deliverable 4 - Exactly Five Presentation Slides 27

Limitations, Ethical Considerations and Conclusion 30

Limitations . . . 30
Ethical considerations . . . 30
Future scope . . . 30
Conclusion . . . 30

References **31**

Deliverable 1 - One-Page Problem Formulation

Motivation and problem. Signalised intersections are a major source of urban delay. Fixed-time plans repeat one cycle regardless of the traffic that actually arrives, so green time is wasted on empty approaches while other approaches queue. The task is to let multiple intelligent agents, one per intersection, adapt signal timing online to cut vehicle waiting time and queue length across a connected network.

Traffic simulation environment. Experiments run through the project's multi-agent environment interface. Backend for the reported numbers: **built-in Python mesoscopic simulator (NOT SUMO)**. One episode is 3600 s of simulated time with an off-peak / peak (1.4x) / off-peak demand profile; routes are generated from Poisson origin-destination flows (8 entry points, 7 destinations each).

Multi-intersection network. A 2 x 2 grid of four signalised intersections (J00, J01, J10, J11) connected by 250 m two-lane links at 13.89 m/s (50 km/h); each intersection has four approaches, so 16 stop-lines and 8 border entry / exit points.

State representation. For every incoming lane the agent observes *density* (vehicles / capacity), *queue length* (halting vehicles / capacity) and *waiting time* (summed waiting seconds / 300), all clipped to [0, 1]; plus a one-hot of the current green phase and the time since the last switch, plus the neighbouring intersections' mean queue and waiting level. State size: 31 values per agent.

Action space. Discrete: pick which *green phase* should be active during the next 10 s decision step (4 phases). Choosing the current phase extends it; choosing another one inserts a 3 s yellow transition first, which gives the safety constraint for free and penalises needless switching through lost time.

Reward function. Local reward r_i = -(0.1 x halting vehicles + sum of waiting seconds / 300) on the agent's incoming lanes, clipped at -10. It is large when queues and waits grow, so maximising return minimises both. Training uses the shared reward (1 - alpha) r_i + alpha x mean(neighbour r_j), alpha = 0.3. Reported cumulative reward is the sum of the *local* rewards of all agents over the episode, so it is comparable between control methods.

Multi-agent RL algorithm. Independent Double DQN: each agent owns a Q-network (obs - 128 - 128 - actions, ReLU), a replay buffer (50000 transitions) and a target network (hard update every 200 learning steps); epsilon-greedy exploration 1.0 to 0.05, Huber loss, Adam (lr 0.0005), gamma 0.9.

Coordination between agents. Agents do not share parameters. They coordinate implicitly through (i) the neighbour queue / waiting summary in the observation, so an agent sees pressure that is about to arrive or is blocking its exit, and (ii) the cooperative reward mix, which makes an agent partly responsible for its neighbours' congestion (shared objective: network-wide waiting time).

Evaluation metrics. On 10 unseen test seeds (demand scale 1.0 and extra scales if listed in the CSV): average waiting time per vehicle (vehicle-seconds standing still / vehicles inserted), average queue length (halting vehicles per controlled lane, sampled every decision step), throughput (vehicles that completed their trip within the hour) and cumulative reward. Fixed-time control uses the same environment with phases cycled 30 s each.

Project Architecture

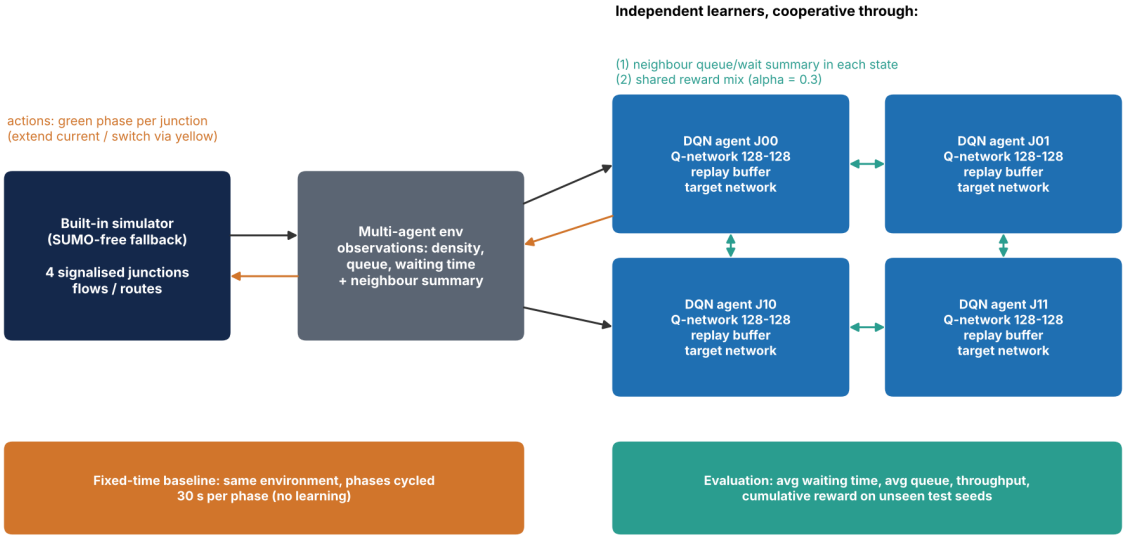


Figure 1 - System architecture: environment, four independent DQN agents with neighbour coordination, fixed-time baseline and evaluation.

Data flow

- Every decision step (10 s) the environment returns one observation per junction; *runner.augment()* appends the neighbours' summary.
- Each agent chooses a green phase epsilon-greedily; the environment applies yellow / green states (TraCI *setRedYellowGreenState* in SUMO) and simulates 10 s.
- Local rewards come back; *runner.shared_rewards()* mixes them cooperatively; every agent stores its transition and performs one Double-DQN update.
- After every few training episodes the greedy policies are run on validation seeds; the best checkpoint is kept and finally tested on unseen seeds against the fixed-time baseline.

Project layout

```
traffic_signal_marl/
  run_all.py            <- full pipeline (train + evaluate + figures + report)
  check_setup.py        <- verify SUMO / TraCI / 4 intersections
  make_report.py        <- builds PDF, PPTX and README from results/
  requirements.txt, README.md
  tsc/
    config.py  demand.py  network.py
    sumo_env.py         <- SUMO + TraCI environment
    minisim_env.py      <- built-in fallback simulator
    runner.py  plots.py
    agents/ dqn_torch.py dqn_numpy.py replay.py __init__.py
  results/              <- generated: CSVs, PNGs, models, PDF, PPTX
  sumo_files/           <- generated: network + route files
```

Simplified or simulated components (read this)

SUMO backend: full microscopic simulation of the 2 x 2 grid; phases are read from the netconvert-generated signal programs. **Built-in backend**: a mesoscopic queue model (1 s step, link travel time, saturation-flow discharge with start-up loss, finite link storage / spill-back) - a deliberate simplification: no car-following, lane changing or driver heterogeneity. Right turns share the through signal group; trips are counted when the vehicle leaves its last junction. **Demand** is synthetic (Poisson OD flows), not measured. **Avg waiting time** = vehicle-seconds with speed below 0.1 m/s divided by vehicles inserted. See the notice on the cover page for which backend produced the reported numbers.

Step-by-Step Execution Guide (VS Code, Windows)

1. Create the folder

Create **C:\Projects\traffic_signal_marl** and extract **Complete_Project_Source_Code.zip** into it so that *run_all.py* sits directly inside. In VS Code choose File - Open Folder - that folder.

2. Open a terminal

Terminal - New Terminal (PowerShell). Check the existing installs (do not reinstall them):

```
python --version        # expect 3.13.2
sumo --version         # expect 1.28.0
echo $env:SUMO_HOME   # should point to the SUMO folder
```

3. Create a virtual environment and install the Python packages

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

If PowerShell blocks activation run *Set-ExecutionPolicy -Scope CurrentUser RemoteSigned* once. If *sumo* is not on PATH set SUMO_HOME, e.g. *$env:SUMO_HOME = "C:\Program Files (x86)\Eclipse\Sumo"*.

4. Verify the setup (all four intersections, both controllers)

```
python check_setup.py
```

5. Run the complete experiment with real SUMO

```
python run_all.py --backend sumo --episodes 60 --test-episodes 10
# optional extras:  --scales 1.0 0.7 1.3   --tune-baseline
```

Everything is written to *results/*: Experiment_Results.csv, summary.csv, training_log.csv, validation_log.csv, Learning_Curves.png, the PDF and PPTX (regenerated from the new SUMO numbers) and the model checkpoints. To rebuild only the documents: *python make_report.py*. To watch SUMO's GUI open *sumo_files\grid2x2.sumocfg* with sumo-gui.

Expected effort: a SUMO episode of 3600 s takes tens of seconds, so 60 episodes plus validation finish in roughly an hour on a laptop; use *--quick* for a 2-minute smoke test.

Deliverable 2 - Complete Python Implementation

The listing below is read from the actual files at build time. *make_report.py* (document generator) is in the ZIP only.

run_all.py

End-to-end pipeline: baseline, training, test evaluation, CSV, figures, report.

```
"""End-to-end pipeline:  environment -> fixed-time baseline -> DQN training -> test
evaluation -> CSV files -> figures -> (optional) PDF/PPTX report.

Examples
    python run_all.py --backend sumo --episodes 60          # real SUMO experiment
    python run_all.py --backend minisim --episodes 150      # built-in simulator (no SUMO)
    python run_all.py --quick                                # tiny smoke test
"""
import argparse
import json
import platform
import sys
import time
from dataclasses import asdict
from pathlib import Path

import numpy as np
import pandas as pd

from tsc.agents import torch_available
from tsc.config import DQNConfig, SimConfig, TrainConfig
from tsc.plots import make_plots
from tsc.runner import evaluate, load_best, make_agents, run_fixed_time, train

ROOT = Path(__file__).resolve().parent


def make_env(backend, cfg, gui=False):
    if backend == "sumo":
        from tsc.sumo_env import SumoEnv
        return SumoEnv(cfg, work_dir=ROOT / "sumo_files", gui=gui)
    from tsc.minisim_env import MiniSimEnv
    return MiniSimEnv(cfg)


def pick_backend(requested):
    if requested != "auto":
        return requested
    try:
        import traci  # noqa: F401
        from tsc.network import find_sumo_binary
        find_sumo_binary("sumo")
        return "sumo"
    except Exception:
        return "minisim"


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--backend", default="auto", choices=["auto", "sumo", "minisim"])
    ap.add_argument("--agent", default="auto", choices=["auto", "torch", "numpy"])
    ap.add_argument("--episodes", type=int, default=None)
    ap.add_argument("--eval-every", type=int, default=None)
    ap.add_argument("--test-episodes", type=int, default=None, help="number of unseen test seeds")
    ap.add_argument("--scales", type=float, nargs="+", default=None, help="demand multipliers for the test")
    ap.add_argument("--tune-baseline", action="store_true", help="also test fixed-time with 20 s and 40 s green")
    ap.add_argument("--out", default=str(ROOT / "results"))
    ap.add_argument("--quick", action="store_true")
    ap.add_argument("--no-report", action="store_true")
    args = ap.parse_args()

    backend = pick_backend(args.backend)
    cfg, hp, tc = SimConfig(), DQNConfig(), TrainConfig()
    if args.quick:
        tc.episodes, tc.eval_every, tc.val_seeds, tc.test_seeds = 4, 2, (900,), (1000, 1001)
        cfg.sim_seconds = 600
        hp.learn_start = 100
    if args.episodes:
        tc.episodes = args.episodes
    if args.eval_every:
        tc.eval_every = args.eval_every
    if args.test_episodes:
        tc.test_seeds = tuple(range(1000, 1000 + args.test_episodes))
    if args.scales:
        tc.test_scales = tuple(args.scales)
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    agent_kind = args.agent if args.agent != "auto" else ("torch" if torch_available() else "numpy")
    print(f"backend={backend}  agent implementation={agent_kind}  episodes={tc.episodes}")

    env = make_env(backend, cfg)
    print("agents:", env.agents, "| actions per agent:", env.n_actions, "| lanes per agent:", env.n_lanes)
    t_start = time.time()

    # ---- fixed-time reference on validation seeds -----------------------------------
```

```
    fv = [run_fixed_time(env, s, 1.0, cfg.fixed_green) for s in tc.val_seeds]
    fixed_val = {k: float(np.mean([r[k] for r in fv])) for k in fv[0]}
    print("fixed-time validation:", {k: round(v, 2) for k, v in fixed_val.items()})

    # ---- training ------------------------------------------------------------
    agents = make_agents(env, hp, agent_kind, seed=0)
    train_log, val_log = train(env, agents, hp, tc, out / "models")
    train_df, val_df = pd.DataFrame(train_log), pd.DataFrame(val_log)
    train_df.to_csv(out / "training_log.csv", index=False)
    val_df.to_csv(out / "validation_log.csv", index=False)
    load_best(agents, out / "models", env)
    best_info = json.loads((out / "models" / "best_info.json").read_text())
    print("best checkpoint:", best_info)

    # ---- final test on unseen seeds ------------------------------------------
    rows = []
    green_options = [cfg.fixed_green] + ([20, 40] if args.tune_baseline else [])
    for scale in tc.test_scales:
        for g in green_options:
            for s in tc.test_seeds:
                m = run_fixed_time(env, s, scale, g)
                rows.append({"controller": f"Fixed-time ({g}s)", "demand_scale": scale, "seed": s, **m})
        for s, m in zip(tc.test_seeds, evaluate(env, agents, tc.test_seeds, scale)):
            rows.append({"controller": "Multi-agent DQN", "demand_scale": scale, "seed": s, **m})
    res = pd.DataFrame(rows)
    res["backend"] = backend
    res["agent_impl"] = agent_kind
    cols = ["controller", "demand_scale", "seed", "avg_wait_s", "avg_queue_per_lane", "avg_total_queue",
            "throughput", "departed", "cum_reward", "backend", "agent_impl"]
    res = res[cols]
    res.to_csv(out / "Experiment_Results.csv", index=False)
    summary = (res.groupby(["controller", "demand_scale"])
               [["avg_wait_s", "avg_queue_per_lane", "avg_total_queue", "throughput", "departed", "cum_reward"]]
               .agg(["mean", "std"]))
    summary.columns = [f"{a}_{b}" for a, b in summary.columns]
    summary = summary.reset_index()
    summary.to_csv(out / "summary.csv", index=False)
    print(summary.round(2).to_string())

    # ---- figures -------------------------------------------------------------
    main_scale = 1.0 if 1.0 in tc.test_scales else tc.test_scales[0]
    sel = res[res.demand_scale == main_scale]
    fx = sel[sel.controller == f"Fixed-time ({cfg.fixed_green}s)"]
    dq = sel[sel.controller == "Multi-agent DQN"]
    label = "SUMO" if backend == "sumo" else "built-in Python simulator (not SUMO)"
    make_plots(out, train_df, val_df, fixed_val, fx.to_dict("list"), dq.to_dict("list"), label)

    meta = {
        "backend": backend, "agent_impl": agent_kind, "python": platform.python_version(),
        "platform": platform.platform(), "main_scale": main_scale, "best_checkpoint": best_info,
        "runtime_s": round(time.time() - t_start, 1), "sim": asdict(cfg), "dqn": asdict(hp),
        "train": {k: (list(v) if isinstance(v, tuple) else v) for k, v in asdict(tc).items()},
        "fixed_val": fixed_val,
    }
    (out / "run_meta.json").write_text(json.dumps(meta, indent=2))
    if hasattr(env, "close"):
        env.close()
    print(f"results written to {out}  ({meta['runtime_s']} s)")
    if not args.no_report:
        try:
            import make_report
            make_report.build(out)
        except ImportError:
            print("make_report.py not found - skipping report")


if __name__ == "__main__":
    sys.exit(main())
```

check_setup.py

Sanity check of SUMO/TraCI/PyTorch; proves all 4 intersections respond.

```
"""Quick sanity check of your SUMO / TraCI / PyTorch installation and the generated
network.  Run this FIRST:   python check_setup.py

It (1) locates SUMO, (2) builds the 2x2 network, (3) starts SUMO through TraCI,
(4) prints every traffic light with its green phases and (5) runs 5 simulated minutes
with the fixed-time controller and with random actions to prove all 4 intersections work.
"""
import os
import sys

import numpy as np


def main():
    print("Python", sys.version.split()[0])
    try:
        import torch
        print("PyTorch", torch.__version__)
    except ImportError:
        print("PyTorch NOT installed -> NumPy DQN fallback will be used (pip install torch)")
    from tsc.network import find_sumo_binary
    print("SUMO_HOME =", os.environ.get("SUMO_HOME"))
```

```
    print("sumo      =", find_sumo_binary("sumo"))
    print("netconvert=", find_sumo_binary("netconvert"))
    from tsc.config import SimConfig
    from tsc.sumo_env import SumoEnv
    cfg = SimConfig()
    cfg.sim_seconds = 300
    env = SumoEnv(cfg, work_dir="sumo_files")
    for a in env.agents:
        print(f"  {a}: {env.n_lanes[a]} incoming lanes, {env.n_actions[a]} green phases")
        for i, g in enumerate(env.greens[a]):
            print(f"      phase {i}: {g}")
    env.reset(seed=1)
    done, n = False, 0
    while not done:
        _, r, done, _ = env.step({a: (n // 3) % env.n_actions[a] for a in env.agents})
        n += 1
    print("fixed-time 300 s :", {k: round(v, 2) for k, v in env.metrics().items()})
    rng = np.random.default_rng(0)
    env.reset(seed=1)
    done = False
    while not done:
        _, r, done, _ = env.step({a: int(rng.integers(env.n_actions[a])) for a in env.agents})
    print("random 300 s     :", {k: round(v, 2) for k, v in env.metrics().items()})
    print("\nOK - all", len(env.agents), "intersections respond to control. You can run run_all.py now.")


if __name__ == "__main__":
    main()
```

tsc/config.py

Grid geometry, demand profile, simulation / reward / DQN / training settings.

```
"""Central configuration: grid geometry, simulation, reward, DQN and training settings."""
from dataclasses import dataclass
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent

# ---------------------------------------------------------------- network layout
# 2 x 2 grid of signalised intersections.  "J<row><col>", row 0 is the top row.
JUNCTIONS = ["J00", "J01", "J10", "J11"]
DIRS = {"N": (-1, 0), "S": (1, 0), "E": (0, 1), "W": (0, -1)}  # (d_row, d_col)
OPP = {"N": "S", "S": "N", "E": "W", "W": "E"}
LEFT = {"N": "W", "W": "S", "S": "E", "E": "N"}  # heading -> heading after a left turn


def junction_rc(j):
    """'J10' -> (1, 0)"""
    return int(j[1]), int(j[2])


def neighbour_map():
    """Adjacent signalised junctions (used for agent coordination)."""
    out = {j: [] for j in JUNCTIONS}
    for a in JUNCTIONS:
        ra, ca = junction_rc(a)
        for b in JUNCTIONS:
            rb, cb = junction_rc(b)
            if abs(ra - rb) + abs(ca - cb) == 1:
                out[a].append(b)
    return out


NEIGHBOURS = neighbour_map()

# ---------------------------------------------------------------- demand
# Entry rate (veh/h) per border entry, depending on the side of the grid it sits on.
ENTRY_RATE_VPH = {"N": 300.0, "S": 300.0, "E": 450.0, "W": 450.0}
# Time-varying demand multiplier: (begin_s, end_s, factor) -> morning-peak style profile.
DEMAND_PROFILE = [(0, 1200, 0.8), (1200, 2400, 1.4), (2400, 3600, 0.8)]


@dataclass
class SimConfig:
    sim_seconds: int = 3600      # one episode = one simulated hour
    delta_time: int = 10         # seconds between two agent decisions
    yellow_time: int = 3         # yellow interval when a phase is changed
    link_length: float = 250.0   # metres between intersections
    free_speed: float = 13.89    # m/s (50 km/h)
    lanes_per_edge: int = 2
    veh_space: float = 7.5       # metres per queued vehicle (length + min gap)
    # reward  r = -(w_queue * halting_vehicles + w_wait * sum_of_waiting_seconds)
    reward_w_queue: float = 0.1
    reward_w_wait: float = 1.0 / 300.0
    reward_floor: float = -10.0
    fixed_green: int = 30        # green time (s) per phase for the fixed-time baseline


@dataclass
class DQNConfig:
    hidden: int = 128
    lr: float = 5e-4
    gamma: float = 0.9
    batch_size: int = 64
```

```
    buffer_size: int = 50000
    target_update: int = 200     # learning steps between hard target-network copies
    learn_start: int = 500       # transitions before learning starts
    eps_start: float = 1.0
    eps_end: float = 0.05
    eps_decay_frac: float = 0.7  # fraction of training episodes over which eps decays
    coop_alpha: float = 0.3      # weight of neighbours' reward in each agent's training reward


@dataclass
class TrainConfig:
    episodes: int = 150
    eval_every: int = 10         # validation every N training episodes
    val_seeds: tuple = (900, 901, 902)
    test_seeds: tuple = tuple(range(1000, 1010))
    train_seed0: int = 1
    train_scale_range: tuple = (0.8, 1.25)  # random demand jitter during training
    test_scales: tuple = (1.0,)
```

tsc/demand.py

Origin-destination demand and routes, shared by both backends.

```
"""Origin-destination demand shared by the SUMO and the built-in simulator backends.

Vehicles enter the 2x2 grid from 8 border entry points and leave through one of the
other border exit points.  Routes are "horizontal first, then vertical".
"""
from .config import (DEMAND_PROFILE, DIRS, ENTRY_RATE_VPH, JUNCTIONS, LEFT, OPP,
                     junction_rc)


def border_points():
    """All (junction, side) pairs whose outer side has no neighbouring junction."""
    pts = []
    for j in JUNCTIONS:
        r, c = junction_rc(j)
        for side, (dr, dc) in DIRS.items():
            if f"J{r + dr}{c + dc}" not in JUNCTIONS:
                pts.append((j, side))
    return pts


def movement_type(h_in, h_out):
    """'T' through, 'L' left, 'R' right (right turns share the through signal group)."""
    if h_in == h_out:
        return "T"
    return "L" if LEFT[h_in] == h_out else "R"


def compute_route(entry, exit_):
    """Route as a list of (junction, heading_in, heading_out, movement) or None if invalid
    (a route that would need a U-turn)."""
    (j0, side0), (j1, side1) = entry, exit_
    r, c = junction_rc(j0)
    tr, tc = junction_rc(j1)
    heading = OPP[side0]
    moves = []
    while (r, c) != (tr, tc):
        if c != tc:
            out = "E" if tc > c else "W"
        else:
            out = "S" if tr > r else "N"
        moves.append([f"J{r}{c}", heading, out])
        r, c = r + DIRS[out][0], c + DIRS[out][1]
        heading = out
    moves.append([f"J{r}{c}", heading, side1])
    for _, h_in, h_out in moves:
        if h_out == OPP[h_in]:
            return None
    return [(j, hi, ho, "L" if movement_type(hi, ho) == "L" else "T") for j, hi, ho in moves]


def od_table():
    """{entry: [(exit, route), ...]} containing every valid origin-destination pair."""
    pts = border_points()
    table = {}
    for e in pts:
        lst = []
        for x in pts:
            route = compute_route(e, x)
            if route is not None:
                lst.append((x, route))
        table[e] = lst
    return table


def rate_at(side, t, scale=1.0):
    """Entry rate (veh/s) at time t for an entry on `side`."""
    for b, e, f in DEMAND_PROFILE:
        if b <= t < e:
            return ENTRY_RATE_VPH[side] * f * scale / 3600.0
    return 0.0
def entry_edge(entry):
    return f"in_{entry[0]}_{entry[1]}"
```

```
def exit_edge(exit_):
    return f"out_{exit_[0]}_{exit_[1]}"
```

tsc/minisim_env.py

Built-in mesoscopic simulator (fallback backend, same interface).

```
"""Built-in lightweight traffic simulator (fallback backend, NO SUMO needed).

IMPORTANT: this is a *simplified mesoscopic* model, not SUMO.  It exists so the whole
pipeline can be executed on machines where SUMO/TraCI is not available.  Results produced
with it must be reported as "built-in simulator", not as SUMO results.

Model summary (1 s time step):
  * 2x2 grid, 4 signalised junctions, each with 4 approaches; every approach has a
    through/right lane group (T) and a left lane group (L)  ->  8 lane groups / junction.
  * Vehicles travel along a link in a fixed free-flow time, then join the stop-line queue
    of their next movement.  A green movement discharges at its saturation flow rate
    after a 2 s start-up loss; yellow time discharges nothing.
  * Links have finite storage: a full downstream link blocks discharge (spill-back).
  * Same phases, decision interval, observation layout and reward as the SUMO backend.
"""
from collections import deque

import numpy as np

from .config import DIRS, JUNCTIONS, NEIGHBOURS, OPP, SimConfig, junction_rc
from .demand import border_points, od_table, rate_at

SIDES = "NESW"
CAP_MV = {"T": 30.0, "L": 15.0}        # nominal storage per lane group (vehicles)
SAT_RATE = {"T": 0.9, "L": 0.45}        # saturation discharge rate (veh/s)
LINK_CAP = 60                           # max vehicles on one link (storage)
STARTUP_LOST = 2                        # s of start-up loss after a phase turns green
CROSSING = 2                            # s to cross the junction
# Four signal phases: (approach sides that are green, movement group)
PHASES = ((("N", "S"), "T"), (("N", "S"), "L"), (("E", "W"), "T"), (("E", "W"), "L"))


class _Approach:
    """One approach (incoming link + stop line) of a junction."""
    __slots__ = ("q", "transit", "n_transit", "credit")

    def __init__(self):
        self.q = {"T": deque(), "L": deque()}
        self.transit = deque()                # (arrival_time, vehicle)
        self.n_transit = {"T": 0, "L": 0}
        self.credit = {"T": 0.0, "L": 0.0}

    @property
    def storage(self):
        return len(self.transit) + len(self.q["T"]) + len(self.q["L"])


class MiniSimEnv:
    backend = "minisim"

    def __init__(self, cfg=None):
        self.cfg = cfg or SimConfig()
        self.agents = list(JUNCTIONS)
        self.n_actions = {a: len(PHASES) for a in self.agents}
        self.n_lanes = {a: 8 for a in self.agents}       # lane groups per agent
        self.entries = border_points()
        self.od = od_table()
        self.link_time = int(round(self.cfg.link_length / self.cfg.free_speed))
        self.total_lanes = sum(self.n_lanes.values())

    # ------------------------------------------------------------------ episode control
    def reset(self, seed=0, scale=1.0):
        self.rng = np.random.default_rng(seed)
        self.scale = scale
        self.t = 0
        self.app = {(j, s): _Approach() for j in self.agents for s in SIDES}
        self.phase = {a: 0 for a in self.agents}
        self.pending = {a: 0 for a in self.agents}
        self.yellow_left = {a: 0 for a in self.agents}
        self.since = {a: 0 for a in self.agents}
        self.backlog = {e: deque() for e in self.entries}
        self.total_wait = 0.0
        self.departed = 0
        self.completed = 0
        self.q_samples = []
        return self._obs()

    def step(self, actions):
        """Apply one green-phase choice per agent and simulate `delta_time` seconds."""
        for a in self.agents:
            if actions[a] != self.phase[a] and self.yellow_left[a] == 0:
                self.pending[a] = actions[a]
                self.yellow_left[a] = self.cfg.yellow_time
        for _ in range(self.cfg.delta_time):
            self._tick()
        stats = {a: self._junction_stats(a) for a in self.agents}
        self.q_samples.append(sum(q for q, _ in stats.values()))
        local = {}
```

```
        for a, (q, w) in stats.items():
            r = -(self.cfg.reward_w_queue * q + self.cfg.reward_w_wait * w)
            local[a] = max(self.cfg.reward_floor, r)
        done = self.t >= self.cfg.sim_seconds
        return self._obs(), local, done, {}

    def metrics(self):
        mean_q = float(np.mean(self.q_samples)) if self.q_samples else 0.0
        return {
            "avg_wait_s": self.total_wait / max(1, self.departed),
            "avg_queue_per_lane": mean_q / self.total_lanes,
            "avg_total_queue": mean_q,
            "throughput": self.completed,
            "departed": self.departed,
        }

    def close(self):
        pass

    # ------------------------------------------------------------------ dynamics
    def _tick(self):
        t = self.t
        self._arrivals(t)
        for ap in self.app.values():
            while ap.transit and ap.transit[0][0] <= t:
                _, veh = ap.transit.popleft()
                mv = veh[0][veh[1]][3]
                ap.n_transit[mv] -= 1
                veh[2] = t
                ap.q[mv].append(veh)
        for a in self.agents:
            self._discharge(a, t)
        self.total_wait += sum(len(ap.q["T"]) + len(ap.q["L"]) for ap in self.app.values())
        self.t += 1

    def _arrivals(self, t):
        for e in self.entries:
            n = self.rng.poisson(rate_at(e[1], t, self.scale))
            for _ in range(n):
                options = self.od[e]
                route = options[int(self.rng.integers(len(options)))][1]
                self.backlog[e].append([route, 0, 0])
            bl = self.backlog[e]
            ap = self.app[(e[0], e[1])]
            while bl and ap.storage < LINK_CAP:
                veh = bl.popleft()
                mv = veh[0][0][3]
                ap.transit.append((t + self.link_time, veh))
                ap.n_transit[mv] += 1
                self.departed += 1

    def _discharge(self, a, t):
        if self.yellow_left[a] > 0:
            self.yellow_left[a] -= 1
            for s in SIDES:
                self.app[(a, s)].credit = {"T": 0.0, "L": 0.0}
            if self.yellow_left[a] == 0:
                self.phase[a] = self.pending[a]
                self.since[a] = 0
            return
        self.since[a] += 1
        sides, green_mv = PHASES[self.phase[a]]
        for s in SIDES:
            ap = self.app[(a, s)]
            for mv in ("T", "L"):
                if s in sides and mv == green_mv and self.since[a] > STARTUP_LOST:
                    ap.credit[mv] += SAT_RATE[mv]
                else:
                    ap.credit[mv] = 0.0
                    continue
                q = ap.q[mv]
                while q and ap.credit[mv] >= 1.0:
                    veh = q[0]
                    route, idx = veh[0], veh[1]
                    if idx == len(route) - 1:            # leaves the network
                        q.popleft()
                        self.completed += 1
                        ap.credit[mv] -= 1.0
                        continue
                    out = route[idx][2]
                    r, c = junction_rc(a)
                    nj = f"J{r + DIRS[out][0]}{c + DIRS[out][1]}"
                    tgt = self.app[(nj, OPP[out])]
                    if tgt.storage >= LINK_CAP:          # spill-back: blocked
                        break
                    q.popleft()
                    veh[1] = idx + 1
                    nmv = route[idx + 1][3]
                    tgt.transit.append((t + self.link_time + CROSSING, veh))
                    tgt.n_transit[nmv] += 1
                    ap.credit[mv] -= 1.0
                ap.credit[mv] = min(ap.credit[mv], 1.0)

    # ------------------------------------------------------------------ observations
    def _junction_stats(self, a):
        """(halting vehicles, summed waiting seconds) over the junction's approaches."""
```

```
        q_tot, w_tot = 0, 0.0
        for s in SIDES:
            for mv in ("T", "L"):
                q = self.app[(a, s)].q[mv]
                q_tot += len(q)
                w_tot += sum(self.t - v[2] for v in q)
        return q_tot, w_tot

    def _obs(self):
        """Per lane group: [density, queue, waiting] (all in 0..1), then one-hot phase and
        time since last phase change."""
        obs = {}
        for a in self.agents:
            f = []
            for s in SIDES:
                ap = self.app[(a, s)]
                for mv in ("T", "L"):
                    q = ap.q[mv]
                    cap = CAP_MV[mv]
                    f += [min(1.0, (len(q) + ap.n_transit[mv]) / cap),
                          min(1.0, len(q) / cap),
                          min(1.0, sum(self.t - v[2] for v in q) / 300.0)]
            onehot = [1.0 if i == self.phase[a] else 0.0 for i in range(len(PHASES))]
            f += onehot + [min(1.0, self.since[a] / 60.0)]
            obs[a] = np.asarray(f, dtype=np.float32)
        return obs
```

tsc/network.py

Builds the SUMO 2x2 network (netconvert) and route files.

```
"""Generate the SUMO road network (2x2 signalised grid) and the route/demand files.

Files are produced with plain XML + `netconvert`, so no SUMO tool-specific templates are
required.  Naming convention:
    junction nodes   J00 J01 J10 J11          (traffic lights, same ids)
    border nodes     B_<junction>_<side>
    entry edges      in_<junction>_<side>      (border  -> junction)
    exit edges       out_<junction>_<side>     (junction -> border)
    inner edges      e_<from>_<to>
"""
import os
import shutil
import subprocess
from pathlib import Path

from .config import DIRS, JUNCTIONS, SimConfig, junction_rc
from .demand import border_points, entry_edge, exit_edge, od_table, rate_at
from .config import DEMAND_PROFILE


def find_sumo_binary(name="sumo"):
    """Locate a SUMO executable (works on Windows with SUMO_HOME or PATH)."""
    exe = name + (".exe" if os.name == "nt" else "")
    home = os.environ.get("SUMO_HOME")
    if home:
        cand = Path(home) / "bin" / exe
        if cand.exists():
            return str(cand)
    found = shutil.which(name)
    if found:
        return found
    raise FileNotFoundError(
        f"Cannot find '{name}'. Set SUMO_HOME (e.g. C:\\Program Files (x86)\\Eclipse\\Sumo) "
        "or add the SUMO bin folder to PATH.")


def _xy(j, cfg):
    r, c = junction_rc(j)
    return c * cfg.link_length, -r * cfg.link_length


def build_network(out_dir, cfg=None):
    """Write nodes/edges XML and run netconvert.  Returns the .net.xml path."""
    cfg = cfg or SimConfig()
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    nodes, edges = [], []
    for j in JUNCTIONS:
        x, y = _xy(j, cfg)
        nodes.append(f'  <node id="{j}" x="{x}" y="{y}" type="traffic_light"/>')
    e_attr = f'numLanes="{cfg.lanes_per_edge}" speed="{cfg.free_speed}"'
    for (j, side) in border_points():
        x, y = _xy(j, cfg)
        dr, dc = DIRS[side]
        bx, by = x + dc * cfg.link_length, y - dr * cfg.link_length
        b = f"B_{j}_{side}"
        nodes.append(f'  <node id="{b}" x="{bx}" y="{by}" type="priority"/>')
        edges.append(f'  <edge id="in_{j}_{side}" from="{b}" to="{j}" {e_attr}/>')
        edges.append(f'  <edge id="out_{j}_{side}" from="{j}" to="{b}" {e_attr}/>')
    for a in JUNCTIONS:
        ra, ca = junction_rc(a)
        for b in JUNCTIONS:
            rb, cb = junction_rc(b)
            if abs(ra - rb) + abs(ca - cb) == 1:
```

```
                edges.append(f'  <edge id="e_{a}_{b}" from="{a}" to="{b}" {e_attr}/>')
    nod = out_dir / "nodes.nod.xml"
    edg = out_dir / "edges.edg.xml"
    nod.write_text('<nodes>\n' + "\n".join(nodes) + '\n</nodes>\n')
    edg.write_text('<edges>\n' + "\n".join(edges) + '\n</edges>\n')
    net = out_dir / "grid2x2.net.xml"
    cmd = [find_sumo_binary("netconvert"), "--node-files", str(nod), "--edge-files", str(edg),
           "--output-file", str(net), "--no-turnarounds", "true", "--no-warnings", "true"]
    subprocess.run(cmd, check=True)
    return str(net)


def write_routes(path, seed=0, scale=1.0):
    """Write a demand file: Poisson flows for every valid origin-destination pair with a
    three-segment (off-peak / peak / off-peak) demand profile.  `seed` only changes the
    SUMO random seed (given on the command line), the file itself is deterministic."""
    table = od_table()
    lines = ['<routes>',
             '  <vType id="car" accel="2.6" decel="4.5" sigma="0.5" length="5" minGap="2.5" '
             'maxSpeed="13.89"/>']
    for seg_i, (b, e, f) in enumerate(DEMAND_PROFILE):
        k = 0
        for entry, options in table.items():
            side = entry[1]
            per_od = rate_at(side, b, scale) * 3600.0 / len(options)   # veh/h per OD pair
            if per_od <= 0:
                continue
            for exit_, _route in options:
                lines.append(
                    f'  <flow id="f{seg_i}_{k}" type="car" begin="{b}" end="{e}" '
                    f'period="exp({per_od / 3600.0:.8f})" from="{entry_edge(entry)}" '
                    f'to="{exit_edge(exit_)}" departLane="best" departSpeed="max"/>')
                k += 1
    lines.append('</routes>')
    Path(path).write_text("\n".join(lines) + "\n")
    return str(path)


def write_sumocfg(path, net, routes):
    """Optional .sumocfg so the scenario can be opened in sumo-gui for visual inspection."""
    Path(path).write_text(
        '<configuration>\n  <input>\n'
        f'    <net-file value="{Path(net).name}"/>\n'
        f'    <route-files value="{Path(routes).name}"/>\n'
        '  </input>\n  <time>\n    <begin value="0"/>\n    <end value="3600"/>\n  </time>\n'
        '</configuration>\n')
    return str(path)
```

tsc/plots.py

Learning-curve and comparison figures from real logs.

```
"""Matplotlib figures built from the real experiment logs (no hand-entered numbers)."""
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

C_DQN, C_FIX = "#1f6fb2", "#d1752b"


def _style(ax, title, xlabel, ylabel):
    ax.set_title(title, fontsize=10, fontweight="bold", loc="left")
    ax.set_xlabel(xlabel, fontsize=8.5)
    ax.set_ylabel(ylabel, fontsize=8.5)
    ax.tick_params(labelsize=8)
    ax.grid(alpha=0.25, lw=0.6)
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)


def _smooth(y, k=10):
    y = np.asarray(y, float)
    if len(y) < k:
        return y
    ker = np.ones(k) / k
    pad = np.concatenate([np.full(k - 1, y[0]), y])
    return np.convolve(pad, ker, mode="valid")


def _bar(ax, fixed, dqn, key, title, ylabel, fmt="{:.1f}"):
    f, d = fixed[key], dqn[key]
    means = [np.mean(f), np.mean(d)]
    stds = [np.std(f), np.std(d)]
    ax.bar([0, 1], means, yerr=stds, color=[C_FIX, C_DQN], width=0.55, capsize=4)
    ax.set_xticks([0, 1])
    ax.set_xticklabels(["Fixed-time", "Multi-agent DQN"], fontsize=8.5)
    for x, m in zip([0, 1], means):
        ax.text(x, m * 0.5, fmt.format(m), ha="center", color="white", fontsize=9, fontweight="bold")
    _style(ax, title, "", ylabel)
    ax.grid(axis="x", visible=False)
def make_plots(out_dir, train_df, val_df, fixed_val, fixed_test, dqn_test, backend_label):
    """train_df/val_df: DataFrames; fixed_val: dict of metric->mean on validation seeds;
```

```
    fixed_test/dqn_test: dict metric -> list of per-seed values on the test seeds."""
    out_dir = Path(out_dir)
    ep = train_df["episode"].values
    val_df = val_df[val_df["episode"] > 0]   # episode 0 = untrained net, off-scale; kept in the CSV

    # --- combined figure ------------------------------------------------------------
    fig, ax = plt.subplots(2, 3, figsize=(15, 8.2))
    a = ax[0, 0]
    a.plot(ep, train_df["cum_reward"], color=C_DQN, alpha=0.25, lw=1)
    a.plot(ep, _smooth(train_df["cum_reward"]), color=C_DQN, lw=2, label="DQN training (10-ep moving avg)")
    a.axhline(np.mean(fixed_test["cum_reward"]), color=C_FIX, ls="--", lw=1.8, label="Fixed-time (test mean)")
    a.legend(fontsize=7.5, frameon=False)
    _style(a, "(a) Training reward per episode", "Training episode", "Cumulative reward (sum of 4 agents)")

    for a, key, title, yl in [(ax[0, 1], "avg_wait_s", "(b) Avg waiting time during training", "Seconds per vehicle"),
                              (ax[0, 2], "avg_queue_per_lane", "(c) Avg queue length during training", "Halting vehicles per lane")]:
        a.plot(val_df["episode"], val_df[key], color=C_DQN, marker="o", ms=3.5, lw=1.8,
               label="Multi-agent DQN (greedy, validation)")
        a.axhline(fixed_val[key], color=C_FIX, ls="--", lw=1.8, label="Fixed-time (validation)")
        a.legend(fontsize=7.5, frameon=False)
        _style(a, title, "Training episode", yl)

    _bar(ax[1, 0], fixed_test, dqn_test, "avg_wait_s", "(d) Average waiting time (test)", "Seconds per vehicle")
    _bar(ax[1, 1], fixed_test, dqn_test, "avg_queue_per_lane", "(e) Average queue length (test)",
         "Halting vehicles per lane", "{:.2f}")
    _bar(ax[1, 2], fixed_test, dqn_test, "throughput", "(f) Vehicle throughput (test)",
         "Vehicles completed / hour", "{:.0f}")
    fig.suptitle(f"Fixed-time vs Multi-Agent DQN  |  backend: {backend_label}  |  mean +/- std over "
                 f"{len(dqn_test['avg_wait_s'])} unseen test seeds", fontsize=11)
    fig.tight_layout(rect=(0, 0, 1, 0.96))
    fig.savefig(out_dir / "Learning_Curves.png", dpi=150)
    plt.close(fig)

    # --- performance across training episodes (4 metrics) ---------------------------
    fig, ax = plt.subplots(1, 4, figsize=(16, 3.6))
    items = [("cum_reward", "Cumulative reward"), ("avg_wait_s", "Avg waiting time (s)"),
             ("avg_queue_per_lane", "Avg queue (veh/lane)"), ("throughput", "Throughput (veh/h)")]
    for a, (k, t) in zip(ax, items):
        a.plot(val_df["episode"], val_df[k], color=C_DQN, marker="o", ms=3, lw=1.8, label="Multi-agent DQN")
        a.axhline(fixed_val[k], color=C_FIX, ls="--", lw=1.8, label="Fixed-time")
        _style(a, t, "Training episode", "")
    ax[0].legend(fontsize=7.5, frameon=False)
    fig.suptitle("Performance across training episodes (greedy policy on validation seeds)", fontsize=10)
    fig.tight_layout(rect=(0, 0, 1, 0.92))
    fig.savefig(out_dir / "performance_across_training.png", dpi=150)
    plt.close(fig)

    # --- comparison bars (4 metrics incl. reward) -----------------------------------
    fig, ax = plt.subplots(1, 4, figsize=(15, 3.8))
    _bar(ax[0], fixed_test, dqn_test, "avg_wait_s", "Avg waiting time", "s / vehicle")
    _bar(ax[1], fixed_test, dqn_test, "avg_queue_per_lane", "Avg queue length", "veh / lane", "{:.2f}")
    _bar(ax[2], fixed_test, dqn_test, "throughput", "Throughput", "veh / hour", "{:.0f}")
    _bar(ax[3], fixed_test, dqn_test, "cum_reward", "Cumulative reward", "sum over 4 agents", "{:.0f}")
    fig.tight_layout()
    fig.savefig(out_dir / "comparison_bars.png", dpi=150)
    plt.close(fig)
```

tsc/runner.py

Fixed-time runner, multi-agent training loop, validation, checkpoints.

```
"""Episode runners: fixed-time baseline, multi-agent DQN training and greedy evaluation.

Coordination between the independent DQN agents happens in two ways (see README):
  1. every agent's observation is extended with the mean queue / waiting level of its
     neighbouring intersections ("neighbour summary"), and
  2. every agent is trained on a *shared* reward that mixes its own reward with the mean
     reward of its neighbours:  r_i = (1 - alpha) * r_i + alpha * mean_j(r_j).
"""
import json
import time
from pathlib import Path

import numpy as np

from .agents import make_agent
from .config import NEIGHBOURS


# ------------------------------------------------------------------ coordination helpers
def augment(env, obs):
    """Append the neighbours' mean (queue, waiting) level to every agent's observation."""
    summ = {}
    for a, o in obs.items():
        lane = o[:3 * env.n_lanes[a]].reshape(-1, 3)
        summ[a] = lane[:, 1:3].mean(axis=0)                  # mean queue, mean waiting
    out = {}
    for a, o in obs.items():
        nb = np.mean([summ[b] for b in NEIGHBOURS[a]], axis=0)
        out[a] = np.concatenate([o, nb]).astype(np.float32)
    return out


def shared_rewards(local, alpha):
    """Mix own reward with the neighbours' mean reward (cooperative reward shaping)."""
```

```
    return {a: (1 - alpha) * r + alpha * float(np.mean([local[b] for b in NEIGHBOURS[a]]))
            for a, r in local.items()}


# ---------------------------------------------------------------- episodes
def run_fixed_time(env, seed, scale=1.0, green_s=30):
    """Fixed-time control: cycle through all green phases, `green_s` seconds each."""
    hold = max(1, int(round(green_s / env.cfg.delta_time)))
    env.reset(seed, scale)
    step, cum = 0, 0.0
    done = False
    while not done:
        acts = {a: (step // hold) % env.n_actions[a] for a in env.agents}
        _, local, done, _ = env.step(acts)
        cum += sum(local.values())
        step += 1
    m = dict(env.metrics())
    m["cum_reward"] = cum
    return m


def run_dqn_eval(env, agents, seed, scale=1.0):
    """Greedy (epsilon = 0) execution of the trained agents."""
    obs = augment(env, env.reset(seed, scale))
    cum, done = 0.0, False
    while not done:
        acts = {a: agents[a].act(obs[a], 0.0) for a in env.agents}
        nobs, local, done, _ = env.step(acts)
        obs = augment(env, nobs)
        cum += sum(local.values())
    m = dict(env.metrics())
    m["cum_reward"] = cum
    return m


def make_agents(env, hp, kind="auto", seed=0):
    probe = augment(env, env.reset(0, 1.0))
    agents = {a: make_agent(len(probe[a]), env.n_actions[a], hp, seed + i, kind)
              for i, a in enumerate(env.agents)}
    if hasattr(env, "close"):
        env.close()
    return agents


def evaluate(env, agents, seeds, scale=1.0):
    return [run_dqn_eval(env, agents, s, scale) for s in seeds]


# ---------------------------------------------------------------- training
def train(env, agents, hp, tc, out_dir, log=print):
    """Train all agents simultaneously.  Returns (train_log, validation_log)."""
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(12345)
    train_log, val_log = [], []
    best_val = float("inf")
    decay_eps = max(1, int(tc.episodes * hp.eps_decay_frac))
    t0 = time.time()

    def validate(ep):
        res = evaluate(env, agents, tc.val_seeds, 1.0)
        row = {"episode": ep}
        for k in res[0]:
            row[k] = float(np.mean([r[k] for r in res]))
        val_log.append(row)
        return row

    row = validate(0)                                       # untrained policy reference
    for ep in range(1, tc.episodes + 1):
        eps = max(hp.eps_end, hp.eps_start - (hp.eps_start - hp.eps_end) * ep / decay_eps)
        scale = float(rng.uniform(*tc.train_scale_range))
        obs = augment(env, env.reset(tc.train_seed0 + ep, scale))
        done, cum_local, cum_shared, losses = False, 0.0, 0.0, []
        while not done:
            acts = {a: agents[a].act(obs[a], eps) for a in env.agents}
            nobs, local, done, _ = env.step(acts)
            nobs = augment(env, nobs)
            shared = shared_rewards(local, hp.coop_alpha)
            for a in env.agents:
                # episode end is a time limit, not a terminal state -> bootstrap (done=0)
                agents[a].store(obs[a], acts[a], shared[a], nobs[a], 0.0)
                loss = agents[a].learn()
                if loss is not None:
                    losses.append(loss)
            obs = nobs
            cum_local += sum(local.values())
            cum_shared += sum(shared.values())
        m = dict(env.metrics())
        m.update(episode=ep, epsilon=eps, demand_scale=scale, cum_reward=cum_local,
                 cum_shared_reward=cum_shared,
                 loss=float(np.mean(losses)) if losses else float("nan"))
        train_log.append(m)
        msg = (f"ep {ep:4d}/{tc.episodes} eps={eps:.2f} scale={scale:.2f} "
               f"reward={cum_local:9.1f} wait={m['avg_wait_s']:6.1f}s "
               f"queue={m['avg_queue_per_lane']:.2f} thr={m['throughput']} "
               f"[{time.time() - t0:.0f}s]")
```

```
        if ep % tc.eval_every == 0 or ep == tc.episodes:
            v = validate(ep)
            msg += f" | VAL wait={v['avg_wait_s']:.1f}s reward={v['cum_reward']:.1f}"
            if v["avg_wait_s"] < best_val:                  # keep best checkpoint (validation seeds)
                best_val = v["avg_wait_s"]
                for a in env.agents:
                    agents[a].save(out_dir / f"best_{a}")
                (out_dir / "best_info.json").write_text(json.dumps({"episode": ep, "val_avg_wait_s": best_val}))
        log(msg)
    for a in env.agents:
        agents[a].save(out_dir / f"final_{a}")
    return train_log, val_log


def load_best(agents, out_dir, env):
    for a in env.agents:
        agents[a].load(Path(out_dir) / f"best_{a}")
```

tsc/sumo_env.py

Multi-agent SUMO environment over TraCI (phases, observations, reward, metrics).

```
"""Multi-agent SUMO environment driven through TraCI.

One agent per traffic light.  Every `delta_time` seconds each agent chooses which GREEN
phase should be active (choosing the current phase = extend it; choosing another phase =
yellow transition followed by that green).  The valid green phases are read from the
traffic-light program that `netconvert` generated, so the code is independent of the exact
signal layout.
"""
import os
import sys
import uuid
from pathlib import Path

import numpy as np

from .config import JUNCTIONS, SimConfig
from .network import build_network, find_sumo_binary, write_routes, write_sumocfg

try:                                    # normal case: `pip install traci`
    import traci
except ImportError:                     # fall back to SUMO's bundled python tools
    if "SUMO_HOME" in os.environ:
        sys.path.append(os.path.join(os.environ["SUMO_HOME"], "tools"))
    import traci                        # noqa: E402  (raises a clear error if still missing)


class SumoEnv:
    backend = "sumo"

    def __init__(self, cfg=None, work_dir="sumo_files", gui=False):
        self.cfg = cfg or SimConfig()
        self.gui = gui
        self.work_dir = Path(work_dir)
        self.work_dir.mkdir(parents=True, exist_ok=True)
        self.net = build_network(self.work_dir, self.cfg)
        write_sumocfg(self.work_dir / "grid2x2.sumocfg", self.net,
                      write_routes(self.work_dir / "routes_nominal.rou.xml", 0, 1.0))
        self.label = "sim_" + uuid.uuid4().hex[:8]
        self.conn = None
        self.agents = list(JUNCTIONS)
        self.n_actions, self.n_lanes = {}, {}
        self._introspect()

    # ------------------------------------------------------------------ SUMO control
    def _start(self, seed, scale):
        self.close()
        routes = write_routes(self.work_dir / f"routes_{self.label}.rou.xml", seed, scale)
        binary = find_sumo_binary("sumo-gui" if self.gui else "sumo")
        cmd = [binary, "-n", self.net, "-r", routes, "--seed", str(seed),
               "--no-step-log", "true", "--no-warnings", "true",
               "--duration-log.disable", "true", "--time-to-teleport", "300"]
        if self.gui:
            cmd += ["--start", "true", "--quit-on-end", "true"]
        traci.start(cmd, label=self.label)
        self.conn = traci.getConnection(self.label)

    def close(self):
        if self.conn is not None:
            try:
                self.conn.close()
            except Exception:
                pass
            self.conn = None

    def _introspect(self):
        """Start SUMO once to discover lanes and green phases of every traffic light."""
        self._start(0, 1.0)
        c = self.conn
        ids = set(c.trafficlight.getIDList())
        missing = [a for a in self.agents if a not in ids]
        if missing:
            raise RuntimeError(f"Traffic lights {missing} not found in the network; got {sorted(ids)}")
        self.lanes, self.lane_cap, self.greens = {}, {}, {}
```

```
        for a in self.agents:
            lanes = list(dict.fromkeys(c.trafficlight.getControlledLanes(a)))
            self.lanes[a] = lanes
            self.n_lanes[a] = len(lanes)
            for ln in lanes:
                self.lane_cap[ln] = max(1.0, c.lane.getLength(ln) / self.cfg.veh_space)
            logic = c.trafficlight.getAllProgramLogics(a)[0]
            greens = []
            for ph in logic.phases:
                s = ph.state
                if ("G" in s or "g" in s) and "y" not in s.lower() and s not in greens:
                    greens.append(s)
            if len(greens) < 2:
                raise RuntimeError(f"Traffic light {a} has fewer than 2 green phases: {logic.phases}")
            self.greens[a] = greens
            self.n_actions[a] = len(greens)
        self.edges = [e for e in c.edge.getIDList() if not e.startswith(":")]
        self.close()

    def reset(self, seed=0, scale=1.0):
        self._start(seed, scale)
        c = self.conn
        self.phase, self.since = {}, {}
        for a in self.agents:
            self.phase[a] = 0
            self.since[a] = 0
            c.trafficlight.setRedYellowGreenState(a, self.greens[a][0])
        self.total_wait, self.departed, self.completed = 0.0, 0, 0
        self.q_samples = []
        self.t = 0
        return self._obs()

    def _sim_second(self):
        c = self.conn
        c.simulationStep()
        self.t += 1
        self.departed += c.simulation.getDepartedNumber()
        self.completed += c.simulation.getArrivedNumber()
        # vehicle-seconds spent standing still (speed < 0.1 m/s) anywhere in the network
        self.total_wait += sum(c.edge.getLastStepHaltingNumber(e) for e in self.edges)

    def step(self, actions):
        c = self.conn
        cfg = self.cfg
        switching = {}
        for a in self.agents:
            if actions[a] != self.phase[a]:
                cur, nxt = self.greens[a][self.phase[a]], self.greens[a][actions[a]]
                yellow = "".join("y" if (x in "Gg" and y not in "Gg") else x
                                 for x, y in zip(cur, nxt))
                c.trafficlight.setRedYellowGreenState(a, yellow)
                switching[a] = actions[a]
        n_yellow = cfg.yellow_time if switching else 0
        for _ in range(n_yellow):
            self._sim_second()
        for a, new in switching.items():
            c.trafficlight.setRedYellowGreenState(a, self.greens[a][new])
            self.phase[a] = new
            self.since[a] = 0
        for _ in range(cfg.delta_time - n_yellow):
            self._sim_second()
        for a in self.agents:
            if a not in switching:
                self.since[a] += cfg.delta_time
            else:
                self.since[a] = cfg.delta_time - n_yellow
        # ---- statistics, reward
        local, total_q = {}, 0
        for a in self.agents:
            q = sum(c.lane.getLastStepHaltingNumber(ln) for ln in self.lanes[a])
            w = sum(c.lane.getWaitingTime(ln) for ln in self.lanes[a])
            total_q += q
            r = -(cfg.reward_w_queue * q + cfg.reward_w_wait * w)
            local[a] = max(cfg.reward_floor, r)
        self.q_samples.append(total_q)
        done = self.t >= cfg.sim_seconds
        obs = self._obs()
        if done:
            self._final = self.metrics()
            self.close()
        return obs, local, done, {}

    def _obs(self):
        """Per incoming lane: [density, queue, waiting] (0..1) then one-hot phase + time
        since last change.  Identical layout to the built-in simulator."""
        c = self.conn
        obs = {}
        for a in self.agents:
            f = []
            for ln in self.lanes[a]:
                cap = self.lane_cap[ln]
                f += [min(1.0, c.lane.getLastStepVehicleNumber(ln) / cap),
                      min(1.0, c.lane.getLastStepHaltingNumber(ln) / cap),
                      min(1.0, c.lane.getWaitingTime(ln) / 300.0)]
            k = self.n_actions[a]
            f += [1.0 if i == self.phase[a] else 0.0 for i in range(k)]
```

```
            f += [min(1.0, self.since[a] / 60.0)]
            obs[a] = np.asarray(f, dtype=np.float32)
        return obs

    def metrics(self):
        if self.conn is None and hasattr(self, "_final"):
            return self._final
        mean_q = float(np.mean(self.q_samples)) if self.q_samples else 0.0
        total_lanes = sum(self.n_lanes.values())
        return {
            "avg_wait_s": self.total_wait / max(1, self.departed),
            "avg_queue_per_lane": mean_q / total_lanes,
            "avg_total_queue": mean_q,
            "throughput": self.completed,
            "departed": self.departed,
        }
```

tsc/agents/__init__.py

Agent factory (PyTorch if installed, else NumPy).

```
"""Agent factory: PyTorch DQN when available, otherwise the NumPy twin."""


def torch_available():
    try:
        import torch  # noqa: F401
        return True
    except ImportError:
        return False


def make_agent(obs_dim, n_actions, hp, seed=0, kind="auto"):
    if kind == "auto":
        kind = "torch" if torch_available() else "numpy"
    if kind == "torch":
        from .dqn_torch import TorchDQNAgent
        return TorchDQNAgent(obs_dim, n_actions, hp, seed)
    from .dqn_numpy import NumpyDQNAgent
    return NumpyDQNAgent(obs_dim, n_actions, hp, seed)
```

tsc/agents/dqn_numpy.py

Identical Double-DQN agent in pure NumPy (fallback).

```
"""Double-DQN agent in pure NumPy (manual back-propagation + Adam).

Fallback used automatically when PyTorch is not installed.  It implements exactly the same
algorithm, network shape (obs -> 128 -> 128 -> actions) and hyper-parameters as
`dqn_torch.py`.
"""
import numpy as np

from .replay import ReplayBuffer


class _MLP:
    def __init__(self, sizes, rng):
        self.W = [rng.normal(0, np.sqrt(2.0 / i), (i, o)).astype(np.float32)
                  for i, o in zip(sizes[:-1], sizes[1:])]
        self.b = [np.zeros(o, np.float32) for o in sizes[1:]]

    def forward(self, x, keep=False):
        acts = [x]
        for k, (W, b) in enumerate(zip(self.W, self.b)):
            x = x @ W + b
            if k < len(self.W) - 1:
                x = np.maximum(x, 0.0)
            acts.append(x)
        return (x, acts) if keep else x

    def backward(self, acts, dout):
        gW, gb = [None] * len(self.W), [None] * len(self.W)
        d = dout
        for k in reversed(range(len(self.W))):
            gW[k] = acts[k].T @ d
            gb[k] = d.sum(0)
            if k > 0:
                d = (d @ self.W[k].T) * (acts[k] > 0)
        return gW, gb

    def copy_from(self, other):
        self.W = [w.copy() for w in other.W]
        self.b = [b.copy() for b in other.b]


class NumpyDQNAgent:
    kind = "numpy"

    def __init__(self, obs_dim, n_actions, hp, seed=0):
        self.hp, self.n_actions = hp, n_actions
        self.rng = np.random.default_rng(seed)
        sizes = [obs_dim, hp.hidden, hp.hidden, n_actions]
        self.q = _MLP(sizes, self.rng)
        self.target = _MLP(sizes, self.rng)
```

```
        self.target.copy_from(self.q)
        self.buffer = ReplayBuffer(hp.buffer_size, obs_dim, seed)
        self.learn_steps = 0
        # Adam state
        self.m = [np.zeros_like(p) for p in self.q.W + self.q.b]
        self.v = [np.zeros_like(p) for p in self.q.W + self.q.b]
        self.t_adam = 0

    def act(self, obs, eps=0.0):
        if self.rng.random() < eps:
            return int(self.rng.integers(self.n_actions))
        return int(np.argmax(self.q.forward(obs[None, :])[0]))

    def store(self, s, a, r, s2, d):
        self.buffer.add(s, a, r, s2, d)

    def learn(self):
        hp = self.hp
        if len(self.buffer) < max(hp.learn_start, hp.batch_size):
            return None
        s, a, r, s2, d = self.buffer.sample(hp.batch_size)
        n = len(s)
        best = np.argmax(self.q.forward(s2), axis=1)                # Double DQN
        q_next = self.target.forward(s2)[np.arange(n), best]
        y = r + hp.gamma * (1.0 - d) * q_next
        out, acts = self.q.forward(s, keep=True)
        q_sa = out[np.arange(n), a]
        err = q_sa - y
        huber = np.where(np.abs(err) <= 1.0, 0.5 * err ** 2, np.abs(err) - 0.5)
        dq = np.clip(err, -1.0, 1.0) / n                             # d(Huber)/d(q_sa)
        dout = np.zeros_like(out)
        dout[np.arange(n), a] = dq
        gW, gb = self.q.backward(acts, dout)
        self._adam(gW + gb)
        self.learn_steps += 1
        if self.learn_steps % hp.target_update == 0:
            self.target.copy_from(self.q)
        return float(huber.mean())

    def _adam(self, grads, b1=0.9, b2=0.999, eps=1e-8):
        self.t_adam += 1
        params = self.q.W + self.q.b
        for i, (p, g) in enumerate(zip(params, grads)):
            self.m[i] = b1 * self.m[i] + (1 - b1) * g
            self.v[i] = b2 * self.v[i] + (1 - b2) * g * g
            mh = self.m[i] / (1 - b1 ** self.t_adam)
            vh = self.v[i] / (1 - b2 ** self.t_adam)
            p -= self.hp.lr * mh / (np.sqrt(vh) + eps)   # in-place update of weights

    def save(self, path):
        np.savez(str(path) + ".npz", **{f"W{i}": w for i, w in enumerate(self.q.W)},
                 **{f"b{i}": b for i, b in enumerate(self.q.b)})

    def load(self, path):
        z = np.load(str(path) + ".npz")
        n = len(self.q.W)
        self.q.W = [z[f"W{i}"] for i in range(n)]
        self.q.b = [z[f"b{i}"] for i in range(n)]
        self.target.copy_from(self.q)
```

tsc/agents/dqn_torch.py

Double-DQN agent in PyTorch (primary implementation).

```
"""Double-DQN agent implemented in PyTorch (primary implementation)."""
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

from .replay import ReplayBuffer


class QNetwork(nn.Module):
    """MLP: obs -> 128 -> 128 -> Q-value per action."""

    def __init__(self, obs_dim, n_actions, hidden):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, n_actions))

    def forward(self, x):
        return self.net(x)


class TorchDQNAgent:
    """One independent learner (own network, target network and replay buffer)."""
    kind = "torch"

    def __init__(self, obs_dim, n_actions, hp, seed=0):
        torch.manual_seed(seed)
        self.hp, self.n_actions = hp, n_actions
        self.rng = np.random.default_rng(seed)
```

```
        self.q = QNetwork(obs_dim, n_actions, hp.hidden)
        self.target = QNetwork(obs_dim, n_actions, hp.hidden)
        self.target.load_state_dict(self.q.state_dict())
        self.opt = torch.optim.Adam(self.q.parameters(), lr=hp.lr)
        self.buffer = ReplayBuffer(hp.buffer_size, obs_dim, seed)
        self.learn_steps = 0

    @torch.no_grad()
    def act(self, obs, eps=0.0):
        """Epsilon-greedy action."""
        if self.rng.random() < eps:
            return int(self.rng.integers(self.n_actions))
        qv = self.q(torch.as_tensor(obs, dtype=torch.float32).unsqueeze(0))
        return int(qv.argmax(dim=1).item())

    def store(self, s, a, r, s2, d):
        self.buffer.add(s, a, r, s2, d)

    def learn(self):
        hp = self.hp
        if len(self.buffer) < max(hp.learn_start, hp.batch_size):
            return None
        s, a, r, s2, d = [torch.as_tensor(x) for x in self.buffer.sample(hp.batch_size)]
        q_sa = self.q(s).gather(1, a.unsqueeze(1)).squeeze(1)
        with torch.no_grad():                       # Double DQN target
            best = self.q(s2).argmax(dim=1, keepdim=True)
            q_next = self.target(s2).gather(1, best).squeeze(1)
            y = r + hp.gamma * (1.0 - d) * q_next
        loss = F.smooth_l1_loss(q_sa, y)
        self.opt.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(self.q.parameters(), 10.0)
        self.opt.step()
        self.learn_steps += 1
        if self.learn_steps % hp.target_update == 0:
            self.target.load_state_dict(self.q.state_dict())
        return float(loss.item())

    def save(self, path):
        torch.save(self.q.state_dict(), str(path) + ".pt")

    def load(self, path):
        self.q.load_state_dict(torch.load(str(path) + ".pt", map_location="cpu"))
        self.target.load_state_dict(self.q.state_dict())
```

tsc/agents/replay.py

Experience replay buffer.

```
"""Uniform experience replay buffer (NumPy), shared by both DQN implementations."""
import numpy as np


class ReplayBuffer:
    def __init__(self, capacity, obs_dim, seed=0):
        self.cap = capacity
        self.s = np.zeros((capacity, obs_dim), np.float32)
        self.s2 = np.zeros((capacity, obs_dim), np.float32)
        self.a = np.zeros(capacity, np.int64)
        self.r = np.zeros(capacity, np.float32)
        self.d = np.zeros(capacity, np.float32)
        self.n = 0
        self.i = 0
        self.rng = np.random.default_rng(seed)

    def add(self, s, a, r, s2, d):
        k = self.i
        self.s[k], self.a[k], self.r[k], self.s2[k], self.d[k] = s, a, r, s2, float(d)
        self.i = (self.i + 1) % self.cap
        self.n = min(self.n + 1, self.cap)

    def sample(self, batch):
        idx = self.rng.integers(0, self.n, size=batch)
        return self.s[idx], self.a[idx], self.r[idx], self.s2[idx], self.d[idx]

    def __len__(self):
        return self.n
```

tsc/__init__.py

Package marker.

```
"""Multi-agent DQN traffic-signal coordination (SUMO + TraCI + PyTorch)."""
```

requirements.txt

```
# Python 3.13 + an existing Eclipse SUMO 1.28 installation (SUMO itself is NOT installed by pip here)
torch>=2.5
numpy>=2.1
pandas>=2.2
matplotlib>=3.9
traci>=1.22          # TraCI client; if pip cannot match 1.28 exactly, the SUMO_HOME\tools copy is used as fallback
# --- only needed for make_report.py (PDF / PPTX generation)
reportlab>=4.2
```

```
python-pptx>=1.0
Pillow>=10.4
```

Training Methodology

All 4 agents are trained simultaneously for 150 episodes of 3600 s. Each episode uses a new random seed and a demand multiplier drawn uniformly from 0.8-1.25, which prevents the agents from memorising one traffic pattern. Epsilon decays linearly to its floor over the first 70% of the episodes. Because episodes end by time limit, the bootstrap target is not cut at the last step.

Data splits: training seeds 1..N (jittered demand), **validation seeds** [900, 901, 902] (greedy policy every 10 episodes, used only to pick the best checkpoint) and **test seeds** 1000-1009 (never used for training or selection).

Hyper-parameter	Value
Network	obs - 128 - 128 - |A|, ReLU
Algorithm	Double DQN, Huber loss, grad-clip 10
Optimiser / learning rate	Adam / 0.0005
Discount gamma	0.9
Batch / replay size	64 / 50000
Target update	every 200 learning steps
Learning starts after	500 transitions / agent
Epsilon	1.0 to 0.05 over 70% of episodes
Cooperative reward weight alpha	0.3
Decision step / yellow	10 s / 3 s
Training episodes	150
Agent implementation	numpy
Training wall-clock	180 s (whole pipeline)
Best checkpoint	episode 140 (validation wait 51.4 s)

Reward and state design rationale

- Queue and waiting terms together penalise both the number of stopped vehicles and how long they have been stopped, which discourages starving a minor approach.
- Choosing the target *phase* instead of toggling gives each agent a small discrete action set and makes 'extend' and 'switch' explicit.
- The neighbour summary and reward mixing implement coordination without sharing network weights; it keeps the learners scalable (observation size independent of network size).

Deliverable 3 - Learning Curves and Result Tables

IMPORTANT: the numbers in this document were produced with the project's built-in Python mesoscopic simulator (tsc/minisim_env.py) because SUMO, TraCI and PyTorch were not available in the environment that built this report. The SUMO code path (tsc/sumo_env.py, tsc/network.py, tsc/agents/dqn_torch.py) is complete but has NOT been executed to produce these figures. Running 'python run_all.py --backend sumo' on a machine with SUMO 1.28 regenerates every table, figure, slide and this PDF from real SUMO runs.

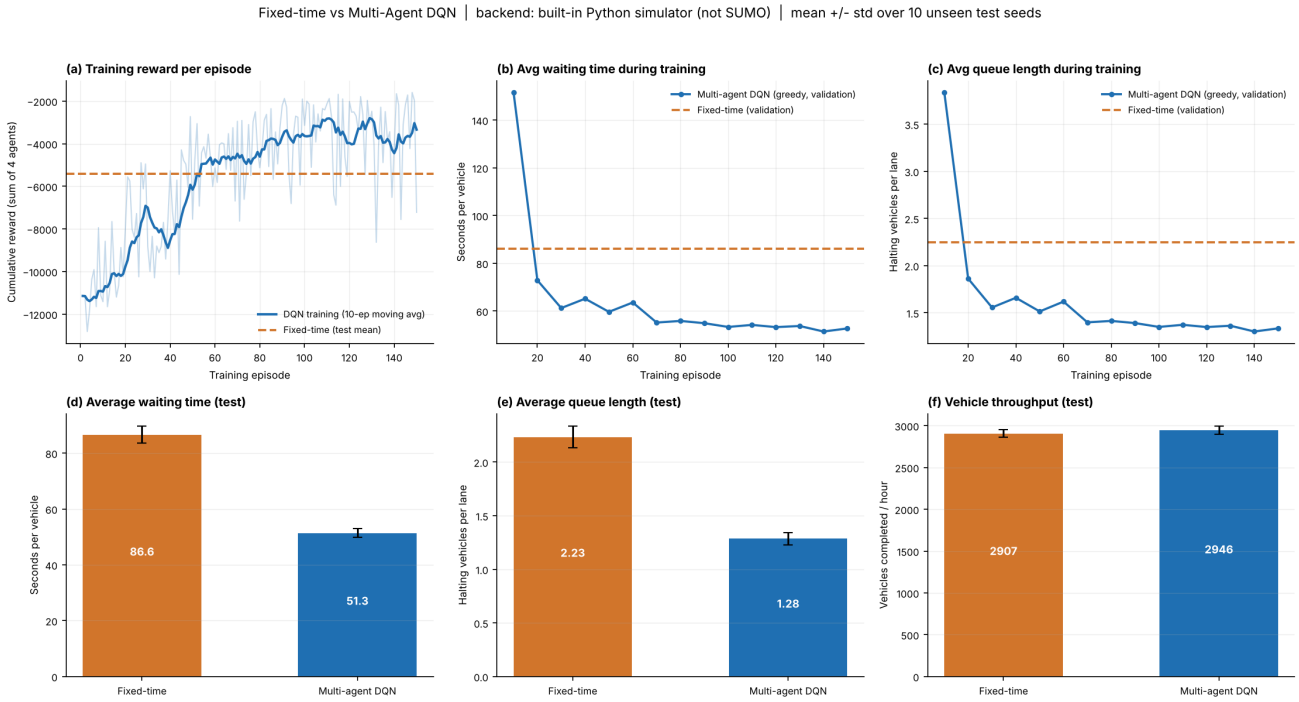


Figure 2 - (a) training reward per episode; (b, c) waiting time and queue length of the greedy policy on validation seeds during training; (d-f) test-seed comparison with fixed-time control. Generated from training_log.csv, validation_log.csv and Experiment_Results.csv.

Table 1 - Test results, demand scale 1.0, mean +/- std over 10 unseen seeds

Metric	Fixed-time (30s)	Multi-agent DQN	Change	DQN better in
Average waiting time (s / vehicle)	86.6 +/- 3.2	51.3 +/- 1.7	-40.7%	10/10 seeds
Average queue length (halting veh / lane)	2.23 +/- 0.11	1.28 +/- 0.06	-42.5%	10/10 seeds
Vehicle throughput (veh completed / hour)	2907 +/- 50	2946 +/- 52	+1.3%	10/10 seeds
Cumulative reward (sum of 4 agents)	-5402 +/- 268	-2570 +/- 159	-52.4%	10/10 seeds

On these test seeds the multi-agent DQN's average waiting time was **40.7% lower** than fixed-time control (51.3 s vs 86.6 s), average queue length **42.5% lower**, throughput **1.3% higher**, and cumulative reward -2570 vs -5402. 'Change' is relative to the fixed-time mean; for waiting time and queue, negative is better.

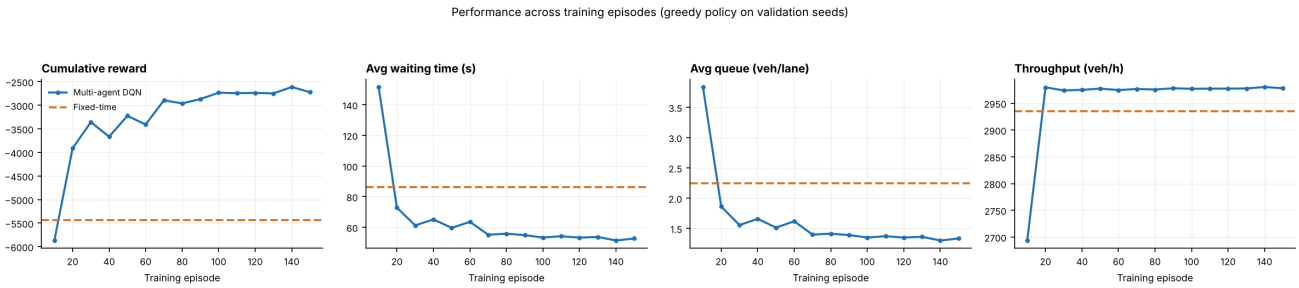


Figure 3 - Performance across training episodes (validation seeds, greedy policy). The untrained episode-0 point is off-scale and omitted here (it is in validation_log.csv).

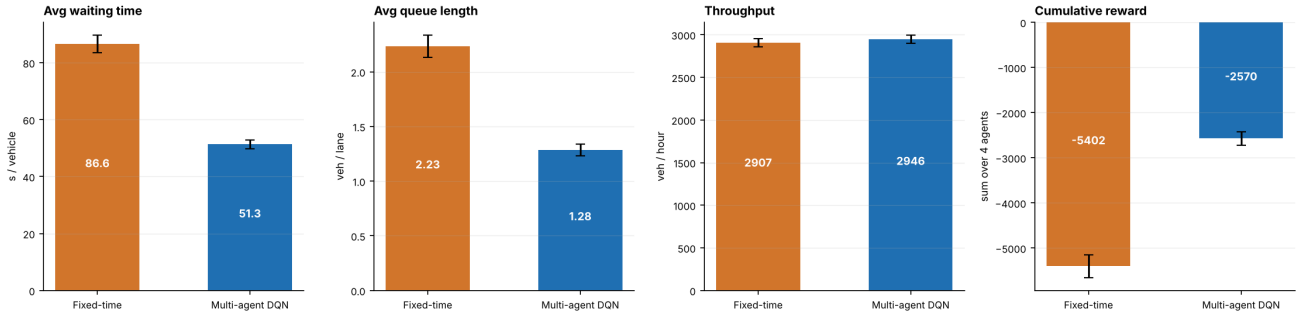


Figure 4 - Fixed-time vs Multi-Agent DQN on the test seeds (mean +/- std).

Table 2 - All evaluated controllers and demand levels (test seeds)

Controller	Demand x	Wait (s)	Queue (veh/lane)	Throughput (veh/h)	Cum. reward
Fixed-time (20s)	0.7	59.7 +/- 1.3	1.06	2036	-2142
Fixed-time (20s)	1	80.5 +/- 6.3	2.07	2922	-4847
Fixed-time (20s)	1.3	165.6 +/- 19.3	5.59	3666	-8790
Fixed-time (30s)	0.7	76.9 +/- 1.3	1.37	2024	-3207
Fixed-time (30s)	1	86.6 +/- 3.2	2.23	2907	-5402
Fixed-time (30s)	1.3	125.5 +/- 12.6	4.23	3761	-9044
Fixed-time (40s)	0.7	94.5 +/- 1.6	1.70	2022	-4466
Fixed-time (40s)	1	101.9 +/- 2.8	2.64	2901	-6940
Fixed-time (40s)	1.3	132.0 +/- 10.0	4.46	3764	-9941
Multi-agent DQN	0.7	39.1 +/- 0.9	0.66	2054	-1216
Multi-agent DQN	1	51.3 +/- 1.7	1.28	2946	-2570
Multi-agent DQN	1.3	293.9 +/- 94.8	7.85	2574	-7718

Table 3 - Per-seed test results (demand scale 1)

Seed	Fixed wait (s)	DQN wait (s)	Fixed queue	DQN queue	Fixed thr.	DQN thr.
1000	85.1	52.1	2.22	1.32	2930	2973
1001	85.2	51.8	2.25	1.33	2981	3020
1002	84.5	49.2	2.15	1.21	2883	2915
1003	86.8	51.7	2.24	1.29	2903	2941
1004	94.1	55.0	2.45	1.40	2943	2977
1005	88.8	50.8	2.35	1.30	2969	3018
1006	88.1	52.3	2.27	1.31	2901	2944
1007	86.2	50.1	2.19	1.24	2882	2915
1008	82.1	49.7	2.08	1.22	2857	2898
1009	85.3	50.6	2.14	1.23	2818	2856

Interpretation

- The learned policy reduced average waiting time by 40.7% and was better than the fixed-time plan on 10 of 10 test seeds.
- Average queue length changed by -42.5% and throughput by +1.3%; throughput is bounded by the demand that is injected in the hour, so large throughput gains are not expected unless the baseline leaves vehicles stuck.
- Fixed-time variants with other green durations were also evaluated (Table 2) to show the baseline is not unfairly weak.

- Robustness across demand levels (waiting time, DQN vs Fixed-time (30s)): 0.7x demand: 39.1 s vs 76.9 s (-49.2%); 1x demand: 51.3 s vs 86.6 s (-40.7%); 1.3x demand: 293.9 s vs 125.5 s (+134.2%). The DQN was trained with demand jitter of 0.8-1.25x, so scales outside that range test generalisation.
- **Failure case:** at 1.3x demand the DQN was worse than fixed-time control and highly variable (std 94.8 s, throughput 2574 vs 3761 veh/h): this over-saturated regime was never seen in training, and the learned policy let queues spill back. Training on a wider demand range (or a pressure-style reward) is the obvious next step.
- Training reward rose from -10942 (first 10 episodes, mostly exploratory) to -3341 (last 10 episodes, epsilon at floor, jittered demand).

Deliverable 4 - Exactly Five Presentation Slides

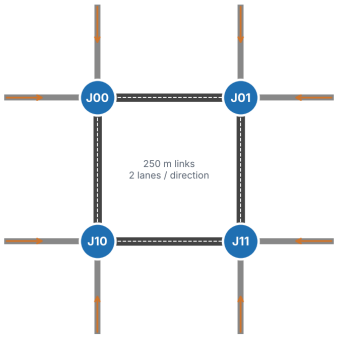
The same five slides are provided as **Five_Slide_Presentation.pptx**.

Slide 1 | Motivation & introduction

Multi-Agent RL for Traffic-Signal Coordination

- Urban congestion wastes travel time, fuel and air quality; much of it is queueing at signals.
- Fixed-time plans repeat the same cycle all day and cannot react to fluctuating demand.
- Neighbouring intersections influence each other: a long queue released upstream arrives downstream seconds later.
- Project 10: one DQN agent per intersection learns to adapt its signal; goal = shorter waits and queues.
- Testbed: 2 x 2 signalised grid, one simulated hour with an off-peak / peak / off-peak demand profile (built-in simulator (SUMO-free fallback)).

2 × 2 grid: 4 signalised junctions, 8 entry / exit points



Traffic-signal coordination | Project 10 | results backend: built-in Python mesoscopic simulator (NOT SUMO) | slide 1 of 5

Slide 2 | Environment, states, actions, rewards

Problem Formulation

- Environment: 4 junctions, 250 m links, 2 lanes per direction, Poisson demand on 8 entries.
- State (per agent): density, queue and waiting time of each incoming lane + one-hot current phase + time since switch + neighbours' mean queue / wait.
- Action: choose the next green phase (10 s decision step): same phase = extend, different = 3 s yellow then switch.
- Reward: r = -(0.1 x halting vehicles + sum of waiting seconds / 300), clipped at -10.
- Training reward is shared: (1 - a) r_i + a mean(r_neighbours), a = 0.3.
- Metrics: avg waiting time, avg queue, throughput, cumulative reward.

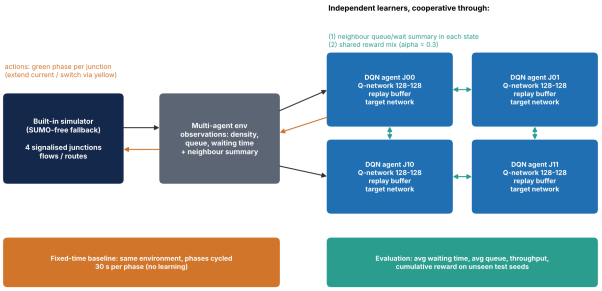
Component	Design
Agents	4 (one per junction)
Actions / agent	10 s step, 4 green phases
State dim / agent	31
Episode	3600 s = 360 decisions

Traffic-signal coordination | Project 10 | results backend: built-in Python mesoscopic simulator (NOT SUMO) | slide 2 of 5

Slide 3 | Architecture & training

Methodology: Multi-Agent Double DQN

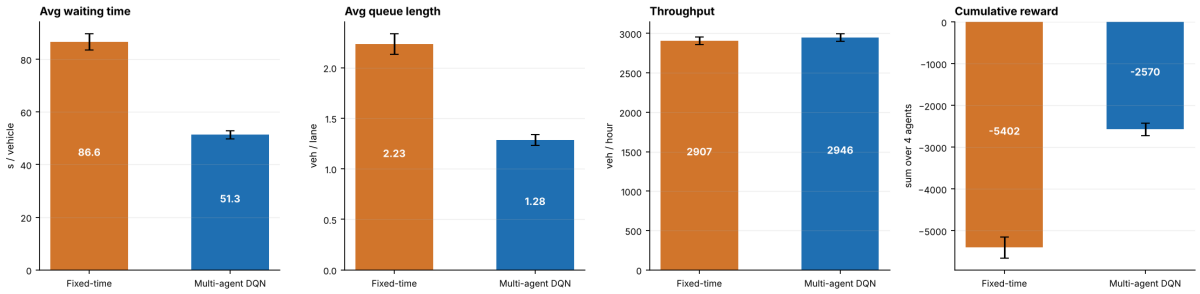
- Independent learners: each junction has its own Q-network (MLP 128-128), replay buffer and target network.
- Double-DQN target, Huber loss, Adam (lr 0.0005), gamma 0.9, batch 64.
- Epsilon-greedy 1.0 -> 0.05; 150 training episodes with random demand jitter.
- Coordination = neighbour summary in the state + cooperative reward mix.
- Model selection on separate validation seeds; final test on unseen seeds against fixed-time control.



Slide 4 | Experimental results (built-in simulator (SUMO-free fallback))

Results: Fixed-Time vs Multi-Agent DQN

- Avg waiting time: 86.6 s -> 51.3 s (40.7% lower with DQN).
- Avg queue: 2.23 -> 1.28 veh/lane (42.5% lower).
- Throughput: 2907 -> 2946 veh/h (1.3% higher).
- Cumulative reward: -5402 -> -2570.
- Mean over 10 unseen test seeds; DQN better on waiting time in 10 of 10 seeds.



Slide 5

Limitations, Ethics, Future Scope & Conclusion

- Limitations: one small grid, synthetic demand, one random-seed family, no pedestrians / buses / sensor noise; independent learners are non-stationary.
- Ethics: safety (min-yellow, human fallback to fixed-time), fairness to minor roads and pedestrians, and equity of who benefits must be audited before deployment.
- Future scope: larger / real networks, graph-based communication (CoLight, MA-PPO), pressure-based rewards, pedestrians, emergency-vehicle priority.
- Conclusion: on this benchmark the multi-agent DQN changed average waiting time by -40.7% and throughput by +1.3% relative to fixed-time control (built-in simulator (SUMO-free fallback)).

Traffic-signal coordination | Project 10 | results backend: built-in Python mesoscopic simulator (NOT SUMO) | slide 5 of 5

Limitations, Ethical Considerations and Conclusion

Limitations

- IMPORTANT: the numbers in this document were produced with the project's built-in Python mesoscopic simulator (tsc/minisim_env.py) because SUMO, TraCI and PyTorch were not available in the environment that built this report. The SUMO code path (tsc/sumo_env.py, tsc/network.py, tsc/agents/dqn_torch.py) is complete but has NOT been executed to produce these figures. Running 'python run_all.py --backend sumo' on a machine with SUMO 1.28 regenerates every table, figure, slide and this PDF from real SUMO runs.
- Small network (4 junctions) and synthetic Poisson demand; no pedestrians, buses, bicycles, detector noise or incidents.
- Independent learners face a non-stationary environment because neighbours also learn; neighbour summary and reward mixing mitigate but do not remove this.
- Only 150 training episodes and one hyper-parameter setting; no seed-wise repetition of the whole training run, so differences between independent training runs are not quantified.
- Average waiting time counts only vehicles that were inserted; vehicles still queued at the end of the hour are not completed trips.

Ethical considerations

- **Safety**: a learned policy must never violate minimum yellow / all-red intervals or pedestrian crossing times; real deployments need a verified safety layer and automatic fallback to a fixed-time plan.
- **Fairness**: minimising average delay can starve minor streets, pedestrians or neighbourhoods; fairness constraints and per-approach maximum waits should be audited.
- **Transparency and accountability**: city engineers need to inspect and override the controller; deep Q-networks are hard to interpret.
- **Data and privacy**: this project only uses simulated, anonymous vehicle counts; real systems based on cameras or connected vehicles must respect privacy law.
- **Equity of benefit**: gains in vehicle throughput should be weighed against emissions, public-transport priority and active-travel safety.

Future scope

- Run on larger and real networks and real demand; calibrate with detector data.
- Graph-based communication (CoLight-style attention), parameter sharing and MA-PPO / QMIX variants.
- Pressure-based rewards (PressLight), pedestrian and public-transport priority, emergency-vehicle pre-emption.
- Multi-seed training repeats and statistical significance tests; robustness to sensor failures.

Conclusion

A multi-agent system of four independent Double-DQN controllers, coordinated through neighbour observations and a cooperative reward, was trained on a 2 x 2 signalised grid and compared with fixed-time control on 10 unseen seeds (built-in Python mesoscopic simulator (NOT SUMO)). The learned controllers outperformed the baseline on average waiting time (51.3 s vs 86.6 s, -40.7%), with queue length -42.5% and throughput +1.3%. The advantage did not hold at every demand level: at 1.3x demand the DQN was worse than fixed-time control. All numbers in this document come directly from the executed experiments stored in the accompanying CSV files. Because the executed backend was the project's simplified simulator, the conclusion applies to that model; rerun with --backend sumo to obtain SUMO-based evidence.

References

[1] Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529-533.

[2] van Hasselt, H., Guez, A., Silver, D. (2016). Deep reinforcement learning with Double Q-learning. *AAAI Conference on Artificial Intelligence*.

[3] Tan, M. (1993). Multi-agent reinforcement learning: independent vs. cooperative agents. *ICML*.

[4] Lopez, P. A. et al. (2018). Microscopic traffic simulation using SUMO. *IEEE International Conference on Intelligent Transportation Systems (ITSC)*.

[5] Genders, W., Razavi, S. (2016). Using a deep reinforcement learning agent for traffic signal control. *arXiv:1611.01142*.

[6] Chu, T., Wang, J., Codeca, L., Li, Z. (2020). Multi-agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, 21(3), 1086-1095.

[7] Wei, H. et al. (2019). PressLight: learning max pressure control to coordinate traffic signals in arterial network. *ACM SIGKDD*.

[8] Wei, H. et al. (2019). CoLight: learning network-level cooperation for traffic signal control. *ACM CIKM*.

[9] Kingma, D. P., Ba, J. (2015). Adam: a method for stochastic optimization. *ICLR*.

[10] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS*.

[11] Eclipse SUMO documentation, TraCI - Traffic Control Interface. https://sumo.dlr.de/docs/